Harness² before vs after: window_aggregates

Date: 2026-11-04 **Benchmark:** WorkBuddy-Bench, code subset **Task:** `feature_pipeline-hard-window_aggregates` **Grading:** the task's hidden tests (13), no LLM judge

1. Result

	Before (base harness)	After (final harness)
Hidden tests passed	**11 / 13**	**13 / 13**
Failed tests	`last event hours u1`, `last event hours u2`	none
Agent steps	19	21
Skill loaded	no	yes, `window-aggregates` (once)
Rollout time	45 s	1 min 53 s

Verdict: On this task, the improved harness fixed exactly the two tests the base failed, and the mechanism is visible in the code the agent wrote (section 4). This is **one run per side**, so it shows the harness *can* cause the fix; it does not yet prove it does so every time.

2. Models and what each did

Role	Model	What it did in this run
Executor (solver)	`sofie-code-0`	Did the task in OpenCode inside the task's Docker container: read the repo, edited `app/window_features.py`, ran it.
Improver	`sofie-think-0` (`medium` effort)	Read the base run's transcript and workspace and edited the harness: INV session, DOM session, then AGG (composition) session.
Grader	hidden tests (not a model)	13 functional tests run after each rollout. The improver never sees them.

All three models are served by the university endpoint `llm.karunya.edu`. `sofie-code-1` was not used in this run. This run followed Google's design: **no test feedback** was given to the improver.

3. The task

- **Source:** WorkBuddy-Bench code subset (Tencent), task `feature_pipeline-hard-window_aggregates`; difficulty *hard*, category *feature-pipeline*, role *algo*; reference change is 1 file, 32 lines.
- **Request (translated from Chinese):** windowed user-event features are unstable at the boundary, and events after the prediction time can leak in. Fix the window aggregation: only use data before `anchor_time`; compute 1-day and 7-day event counts, amounts, and the time since the most recent event; an event exactly at `anchor_time` must not count.
- **Why this task:** in a 16-task screen it was one the base agent *nearly* solves (11/13), where a harness hint can tip the result.

4. Why “after” passed: the evidence

Base agent’s code (from its patch):

```
features["last_event_hours"] = int(time_diff.total_seconds() / 3600)
```

`int()` rounds the gap down to whole hours, so a 1-second gap becomes 0. That is exactly the two failed tests (`last event hours u1/u2`).

The final harness’s skill says: > last_event_hours = (anchor_time - latest strictly-earlier event).total_seconds() / 3600, emitted as exact fractional hours. Never int(), floor() or round() to whole hours.

After agent’s code (from its patch):

```
last_event_hours = (anchor_time - latest_event_time).total_seconds() / 3600
```

The agent loaded the skill and wrote the exact form it prescribes. All 13 tests passed.

5. What the improver changed in the harness

Stage	Files changed	Tests on its trial run
INV	new skill `verify-window-aggregates` (hand-check outputs against the raw data) + system-prompt trigger	6 / 13
DOM	new skill `window-aggregates` (window rules, boundary rows) + guardrail	3 / 13
AGG (final)	merged skill `window-aggregates` + one system-prompt trigger line	**13 / 13**

The final skill has four parts: (1) treat the repo README as the contract; implement `build()` only, don’t change the CLI; (2) rules: window is `anchor − W ≤ t < anchor`, exact fractional hours, one row per anchor; (3) verify by hand-computing the boundary rows from the raw data, including leakage and rounding checks; (4) finish gate: run the documented CLI and reread the diff.

The boundary rows in the skill (for example *u1 has an event exactly at anchor_time, which must be excluded*) are **facts about this workspace’s data**, found from the base run. Google’s method encourages this (“verified facts about this workspace”), but it means this harness is tailored to this task.

6. Caveats

1. **One run per side.** The base has scored 11/13 in both of its runs (screen and pipeline). The after-harness has one run. Repeats are needed to show the 13/13 is stable.
2. **Both trial candidates scored worse than the base (6 and 3).** The AGG step composed a harness better than either candidate. I have not analysed why the candidates hurt. A likely cause is that each rollout is a single random draw.
3. **The harness is task-specific.** It shows per-task adaptation working, not a general improvement for other tasks.
4. **The rest of the run:** `stream_release` went from 9 to 7 (its base varies between 7 and 9). `fallback_signers` had no final run, because the laptop restarted at 16:45.

7. Cost and time

- `think-0` sessions for the 3 tasks ran 21 to 95 minutes each (audit logs). The pipeline allows 2 at a time, which made the run about 2.5 hours.
- Budget at the end of the day: `code-0` 1.89 of 2.5, `think-0` 1.45 of 3.0, covering all of today's runs including the 16-task screen.

8. Next step to make this proof

Run the base and the final harness **3 times each** on this task (6 rollouts, about 0.4 of `code-0`). If the final harness scores 13/13 consistently while the base stays at 11/13, that is a demonstrated improvement on this task.